# AGRILINK: SYSTEM ANALYSIS AND DESIGN DOCUMENT

## PROJECT TITLE:

**A Smart Agricultural Marketplace with Integrated Extension and Nutrition Advisory Services**

Group members:

Mwiwa Yusuf

Nalule Aisha

Nakiranda Hafswah ·

## 1. Problem Statement

Small-scale farmers in Uganda grow food but often cannot reach fair markets directly. They depend on middlemen who buy at low prices and resell at much higher ones, so the farmer carries the risk and the labor while the middleman keeps much of the profit.

Buyers such as households, one by one customer restaurants and small traders also struggle to find fresh produce from a known farmer at a clear fixed price.

A second problem comes beside the first one Many farmers and ordinary households have little access to agricultural extension advice (how to grow crops well, keep animals healthy, and handle produce after harvest and all other agriculture related services such as apiculture, value addition, to mention but a few) and to basic nutrition education. Extension officers are few and far apart, and good advice rarely reaches people on their phones.

AgriLink is a Flutter mobile app with its own FastAPI backend that addresses both problems. Farmers list their produce with quantity, price and location. Buyers browse the listings and place orders directly with the farmer. Every user can read advisory articles on crops, livestock and nutrition written by administrators as well as a direct communication between the farmers and the advisory agents. By removing the middleman and putting trusted advice in the same app, AgriLink helps farmers earn more and helps the public eat and farm better.

AgriLink serves three kinds of buyer: traders who buy in bulk, (schools)businesses such as hotels and restaurants, and ordinary households. It also serves urban farmers, for example horticulture growers, whose fresh produce spoils quickly (such as the nakati sellers,buga etc.). These farmers can mark a listing as urgent so that it is shown first and sold before it rots.

## 2. Users and Their Goals

| User | Who they are | What they want to do |
|---|---|---|
| Farmer | A producer of crops, animals, honey, rabbits or farm inputs. Type: smallholder, urban or commercial | Register, list produce with a price, minimum order and harvest date, mark urgent produce, answer orders, message buyers, ask the extension officer, read advice |
| Buyer: trader | A bulk or international buyer | Find large quantities, filter by minimum order, order directly from farmers |
| Buyer: business | A hotel, restaurant or shop | Find regular fresh supply, order, message the farmer |
| Buyer: household | A person buying for the home | Find fresh produce in small quantities, see urgent and fresh offers, order |
| Administrator | An agriculture or nutrition professional (extension officer) | Publish advisory articles, maintain the product catalogue, answer farmers' and consumers questions on nutrition questions. |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

## 3. Requirements

### 3.1 Functional Requirements

**FR1.** The system shall allow a new user to register with full name, phone number, email, password and role (farmer or buyer). Buyers also choose a buyer type (trader, business or household) and farmers choose a farmer type (smallholder, urban or commercial).

**FR2.** The system shall allow a registered user to log in and shall return an access token.

**FR3.** The system shall store passwords only in hashed form (bcrypt).

**FR4.** The system shall allow a logged-in user to view and edit their own profile.

**FR5.** The system shall allow a farmer to create a produce listing with a product, quantity, unit price, minimum order quantity, location, description, harvest date and best-before date.

**FR6.** The system shall allow a farmer to mark a listing as urgent so that it is shown first in the Urgent / Fresh Today section.

**FR7.** The system shall allow a farmer to view, edit and delete only their own listings.

**FR8.** The system shall allow any logged-in user to browse available listings and filter them by product, location, minimum order quantity and urgency.

**FR9.** The system shall allow a buyer to place an order against a listing for a chosen quantity.

**FR10.** The system shall reject an order whose quantity is below the listing's minimum order quantity or above the quantity available.

**FR11.** The system shall allow a buyer to view their own orders and to cancel an order that is still pending.

**FR12.** The system shall allow a farmer to view orders placed on their listings and to accept or reject them.

**FR13.** The system shall reduce the available quantity of a listing when the farmer accepts an order.

**FR14.** The system shall allow a user to send a message to another user, optionally about a specific listing.

**FR15.** The system shall allow a user to view their inbox and each conversation, and to edit or delete only messages they sent.

**FR16.** The system shall allow a farmer to send a question to an administrator (extension officer) and the administrator to reply in the same conversation.

**FR17.** The system shall allow an administrator to create, edit and delete advisory articles in the categories crops, livestock, apiculture, rabbits, fisheries, farm inputs and nutrition.

**FR18.** The system shall allow any logged-in user to read advisory articles and filter them by category.

**FR19.** The system shall allow an administrator to maintain the product catalogue (add, edit, delete products such as maize, honey, rabbits or manure).

**FR20.** The system shall return 401 for any protected request with a missing or invalid token.

**FR21.** The system shall allow a user to log out, after which the app shall delete the stored token.

3.2 Non-Functional Requirements

**NFR1 (Usability on slow connections).** The app shall show a loading indicator for every request and shall remain usable on a slow mobile connection, loading lists in pages of 20 records or fewer.

**NFR2 (Security).** Passwords shall be hashed, tokens shall expire after 60 minutes, and a user shall never be able to read or change another user's private records.

**NFR3 (Reliability).** When the server cannot be reached, the app shall show a clear error message with a retry option instead of crashing.

**NFR4 (Performance).** Common API requests (login, list listings) shall respond in under 2 seconds on a laptop-hosted server with typical demo data.

**NFR5 (Maintainability).** The Flutter code shall be organised into models, services, screens and widgets, and the database shall match the ERD exactly.

**NFR6 (Portability).** The backend shall run on a laptop by following the steps in its README, and the app shall run on Android phones and emulators.

## 4. User Stories

As a **farmer**, I want to register and log in so that I can manage my produce securely.

As a **farmer**, I want to list my maize with a price and quantity so that buyers can find it without a middleman.

As an **urban farmer**, I want to mark my vegetables as urgent so that they are sold before they rot.

As a **farmer**, I want to set a minimum order so that bulk traders and households are matched to the right listings.

As a **farmer**, I want to edit the price or quantity of a listing so that it matches what I still have.

As a **farmer**, I want to delete a listing once I have sold out so that buyers do not order what is gone.

As a **farmer**, I want to see incoming orders and accept or reject them so that I decide who I sell to.

As a **trader**, I want to filter listings by minimum order so that I can find large quantities quickly.

As a **household buyer**, I want to see the Urgent / Fresh Today section so that I can buy fresh produce at good prices.

As a **business buyer**, I want to order a quantity from a listing so that I get a regular supply directly from the farmer.

As a **buyer**, I want to see my orders and cancel a pending one so that I stay in control of what I buy.

As a **buyer or farmer**, I want to message the other person about a listing so that we can agree on price and pickup.

As a **farmer**, I want to ask the extension officer a question so that I can solve a problem on my farm.

As an **administrator**, I want to reply to farmers' questions so that they get expert help.

As any **user**, I want to read advice on crops, livestock, bees, rabbits and nutrition so that I can farm and eat better.

As an **administrator**, I want to publish and edit advisory articles so that the advice stays accurate and current.

As an **administrator**, I want to maintain the product catalogue so that listings use consistent names and units.

As any **user**, I want to log out so that nobody else can use my account on my phone.

## 5. Entity Relationship Diagram (ERD)

5.1 Entities and Relationships

The database has six entities: **Users, Crops, Produce Listings, Orders, Messages and Advisory**.

Crops is a separate table (it holds any product, including honey, rabbits and farm inputs) so that names and units are stored once and not repeated in every listing. This keeps the design in third normal form.

| Relationship | Cardinality | Implemented by |
|---|---|---|
| One User (farmer) has many Produce Listings | 1:N | `Produce_Listings.farmer_id` references `Users.id` |
| One Crop appears in many Produce Listings | 1:N | `Produce_Listings.crop_id` references `Crops.id` |
| One User (buyer) places many Orders | 1:N | `Orders.buyer_id` references `Users.id` |
| One Produce Listing receives many Orders | 1:N | `Orders.listing_id` references `Produce_Listings.id` |
| One User sends many Messages | 1:N | `Messages.sender_id` references `Users.id` |
| One User receives many Messages | 1:N | `Messages.receiver_id` references `Users.id` |
| One Produce Listing may have many Messages about it | 1:N | `Messages.listing_id` references `Produce_Listings.id` (optional) |
| One User (administrator) writes many Advisory articles | 1:N | `Advisory.author_id` references `Users.id` |

There are no many-to-many relationships in this version. Each order is for one listing. If the project later needs one order to contain several listings, an order items junction table would resolve that many-to-many relationship between Orders and Produce Listings.

### 5.2 Normalization

**1NF:** Every field holds one value and every table has a primary key.

**2NF:** All primary keys are single columns (`id`), so no field depends on part of a key.

**3NF:** Non-key fields depend only on the key. Crop name and unit live in Crops, not in Produce Listings. Farmer details live in Users, not in listings or orders. The order's total price is stored at the time of ordering so that later price changes in the listing do not rewrite past orders.

### 5.3 ERD tables

```
Table users {
  id integer [pk, increment]
  full_name varchar(100) [not null]
  phone varchar(20) [not null, unique]
  email varchar(120) [not null, unique]
  password_hash varchar(255) [not null]
  role varchar(10) [not null, note: 'farmer, buyer or admin']
  farmer_type varchar(15) [note: 'smallholder, urban or commercial; farmers only']
  buyer_type varchar(15) [note: 'trader, business or household; buyers only']
  district varchar(60)
  created_at datetime [not null]
}
```

```
Table crops {
  id integer [pk, increment]
  name varchar(80) [not null, unique]
  category varchar(30) [not null, note: 'cereal, legume, vegetable, fruit, livestock, apiculture, rabbits, fisheries, farm_inputs']
  unit varchar(20) [not null, note: 'kg, bunch, litre, tray, bag']
}

Table produce_listings {
  id integer [pk, increment]
  farmer_id integer [not null, ref: > users.id]
  crop_id integer [not null, ref: > crops.id]
  quantity decimal(10,2) [not null]
  price_per_unit decimal(12,2) [not null]
  min_order_qty decimal(10,2) [not null, note: 'smallest quantity a buyer may order']
  location varchar(100) [not null]
  description text
  harvest_date date
  best_before date
  is_urgent boolean [not null, note: 'true = shown in Urgent / Fresh Today']
  status varchar(15) [not null, note: 'available or sold_out']
  created_at datetime [not null]
}
```

```
Table orders {
  id integer [pk, increment]
  buyer_id integer [not null, ref: > users.id]
  listing_id integer [not null, ref: > produce_listings.id]
  quantity decimal(10,2) [not null]
  total_price decimal(14,2) [not null]
  status varchar(15) [not null, note: 'pending, accepted, rejected, cancelled']
  created_at datetime [not null]
}

Table messages {
  id integer [pk, increment]
  sender_id integer [not null, ref: > users.id]
  receiver_id integer [not null, ref: > users.id]
  listing_id integer [ref: > produce_listings.id, note: 'optional']
  body text [not null]
  created_at datetime [not null]
}

Table advisory {
  id integer [pk, increment]
  author_id integer [not null, ref: > users.id]
  title varchar(150) [not null]
  category varchar(20) [not null, note: 'crops, livestock, apiculture, rabbits,
fisheries, farm_inputs or nutrition']
```

body text [not null]

created_at datetime [not null]

}

## ER DAIGRAM

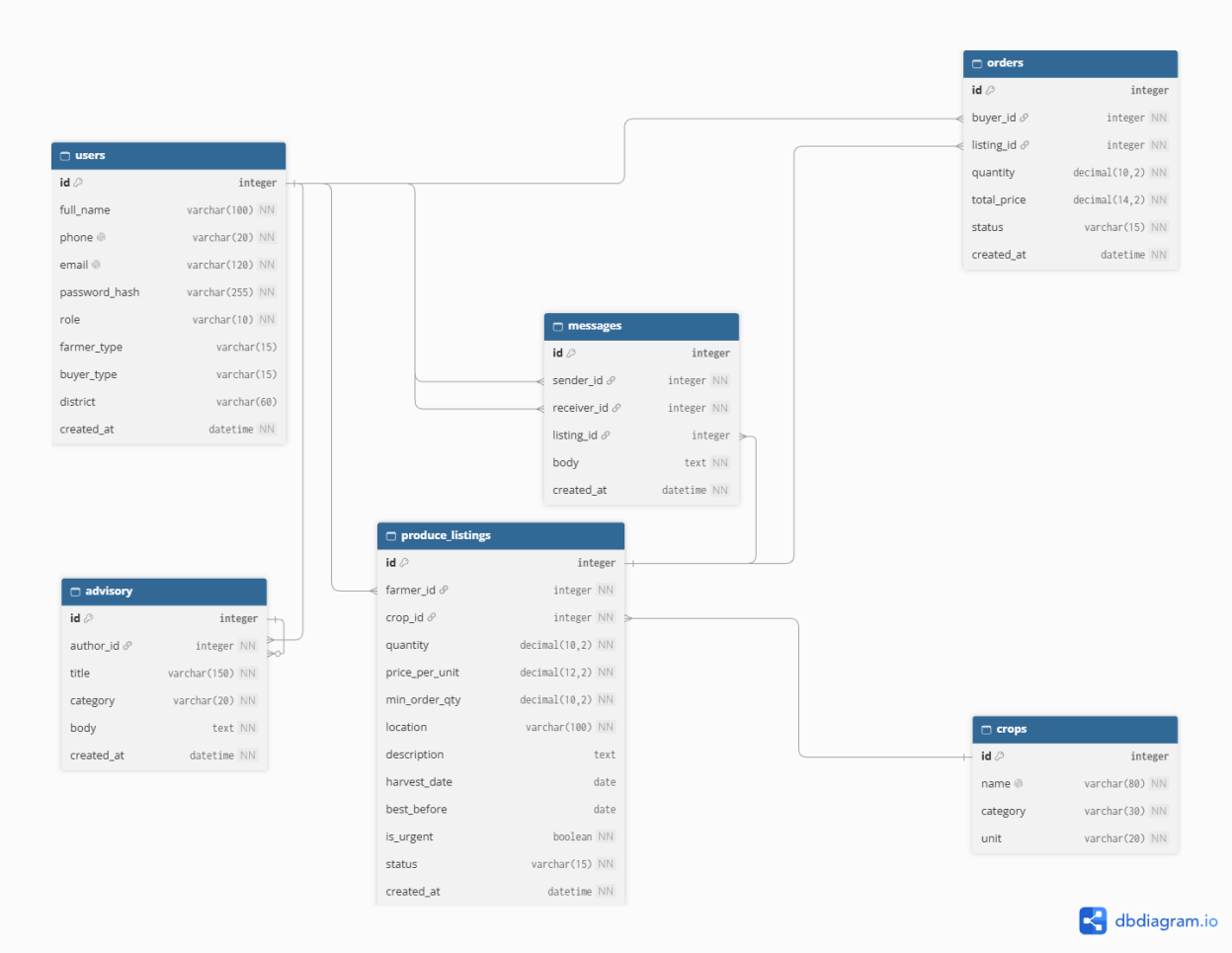



Figure 1: AgriLink ERD

## 6. Data Dictionary

### 6.1 Users

| Field | Type | Size | Constraints | Meaning |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique user number |
| full_name | Varchar | 100 | Not null | User's full name |
| Phone | Varchar | 20 | Not null, unique | Phone number, used for contact |
| Email | Varchar | 120 | Not null, unique | Login email |
| password_hash | Varchar | 255 | Not null | bcrypt hash of the password |
| Role | Varchar | 10 | Not null, one of farmer/buyer/admin | What the user can do |
| farmer_type | Varchar | 15 | Optional, smallholder/urban/commercial | Kind of farmer (farmers only) |
| buyer_type | Varchar | 15 | Optional, trader/business/household | Kind of buyer (buyers only) |
| District | Varchar | 60 | Optional | Where the user is based |
| created_at | Datetime | - | Not null | Registration time |

### 6.2 Crops

| Field | Type | Size | Constraints | Meaning |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique product number |
| Name | Varchar | 80 | Not null, unique | Product name (e.g. Maize, Honey, Rabbit, Manure) |
| category | Varchar | 30 | Not null | cereal, legume, vegetable, fruit, livestock, apiculture, rabbits, fisheries or farm_inputs |
| Unit | Varchar | 20 | Not null | Unit it is sold in (kg, bunch, litre, tray, bag) |

### 6.3 Produce_Listings

| Field | Type | Size | Constraints | Meaning |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique listing number |
| farmer_id | Integer | - | FK to Users.id, not null | Farmer who owns the listing |
| crop_id | Integer | - | FK to Crops.id, not null | What is being sold |
| Quantity | Decimal | 10,2 | Not null, greater than 0 | Amount still available |
| price_per_unit | Decimal | 12,2 | Not null, greater than 0 | Price in UGX per unit |
| min_order_qty | Decimal | 10,2 | Not null, greater than 0 | Smallest quantity a buyer may order (bulk or household) |
| Location | Varchar | 100 | Not null | Where the produce can be collected |
| Description | Text | - | Optional | Quality and notes |
| harvest_date | Date | - | Optional | When it was harvested |
| best_before | Date | - | Optional | Last day it is good to sell or eat |
| is_urgent | Boolean | - | Not null, default false | True = shown in Urgent / Fresh Today |
| Status | Varchar | 15 | Not null, available/sold_out | Whether it can be ordered |
| created_at | Datetime | - | Not null | When it was listed |

### 6.4 Orders

| Field | Type | Size | Constraints | Meaning |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique order number |
| buyer_id | Integer | - | FK to Users.id, not null | Buyer who placed the order |
| listing_id | Integer | - | FK to Produce_Listings.id, not null | Listing being ordered |
| quantity | Decimal | 10,2 | Not null, greater than 0 | Amount ordered |
| total_price | Decimal | 14,2 | Not null | quantity times price at order time (UGX) |
| Status | Varchar | 15 | Not null, pending/accepted/rejected/cancelled | Order progress |
| created_at | Datetime | - | Not null | When it was placed |

### 6.5 Advisory

| **Field** | **Type** | **Size** | **Constraints** | **Meaning** |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique article number |
| author_id | Integer | - | FK to Users.id, not null | Administrator who wrote it |
| Title | Varchar | 150 | Not null | Article title |
| category | Varchar | 20 | Not null | crops, livestock, apiculture, rabbits, fisheries, farm_inputs or nutrition |
| Body | Text | - | Not null | Full advice text |
| created_at | Datetime | - | Not null | Publication time |

### 6.6 Messages

| Field | Type | Size | Constraints | Meaning |
|---|---|---|---|---|
| Id | Integer | - | PK, auto-increment | Unique message number |
| sender_id | Integer | - | FK to Users.id, not null | Who sent the message |
| receiver_id | Integer | - | FK to Users.id, not null | Who receives it |
| listing_id | Integer | - | FK to Produce_Listings.id, optional | Listing the message is about |
| body | Text | - | Not null | The message text |
| created_at | Datetime | - | Not null | When it was sent |

## 7. API Endpoint Table

All endpoints return JSON. Every endpoint except register and login requires the header `Authorization: Bearer <token>` and returns **401** if it is missing or invalid. Wrong input returns **400**, a missing record returns **404**, successful reads and updates return **200**, and successful creation returns **201**.

| Method | Path | Purpose | Request body | Response |
|---|---|---|---|---|
| POST | /auth/register | Create an account | full_name, phone, email, password, role, farmer_type or buyer_type | 201 user (no password) |
| POST | /auth/login | Log in | email, password | 200 access_token, token_type, user |
| GET | /users/me | View own profile | none | 200 user |
| PUT | /users/me | Edit own profile | full_name, phone, district | 200 user |

| | | | | |
|---|---|---|---|---|
| GET | /crops | List products | none | 200 list of products |
| POST | /crops | Add a product (admin) | name, category, unit | 201 product |
| PUT | /crops/{id} | Edit a product (admin) | name, category, unit | 200 product |
| DELETE | /crops/{id} | Delete a product (admin) | none | 200 message |
| GET | /listings | Browse available listings; filters: crop_id, location, max_min_order, urgent=true | query parameters | 200 list of listings (urgent first) |
| GET | /listings/mine | Farmer's own listings | none | 200 list of listings |
| GET | /listings/{id} | One listing | none | 200 listing |
| POST | /listings | Create a listing (farmer) | crop_id, quantity, price_per_unit, min_order_qty, location, description, harvest_date, best_before, is_urgent | 201 listing |
| PUT | /listings/{id} | Edit own listing | any listing field above | 200 listing |
| DELETE | /listings/{id} | Delete own listing | none | 200 message |
| POST | /orders | Place an order (buyer) | listing_id, quantity | 201 order |
| GET | /orders/mine | Buyer's own orders | none | 200 list of orders |

| | | | | |
|---|---|---|---|---|
| GET | /orders/received | Orders on the farmer's listings | none | 200 list of orders |
| PUT | /orders/{id}/status | Farmer accepts or rejects an order | status | 200 order |
| DELETE | /orders/{id} | Buyer cancels a pending order | none | 200 message |
| POST | /messages | Send a message | receiver_id, listing_id (optional), body | 201 message |
| GET | /messages/inbox | List my conversations | none | 200 list of conversations |
| GET | /messages/with/{user_id} | Read one conversation | none | 200 list of messages |
| PUT | /messages/{id} | Edit a message I sent | body | 200 message |
| DELETE | /messages/{id} | Delete a message I sent | none | 200 message |
| GET | /advisory | List articles, filter by category | query parameter | 200 list of articles |
| GET | /advisory/{id} | Read one article | none | 200 article |
| POST | /advisory | Publish an article (admin) | title, category, body | 201 article |
| PUT | /advisory/{id} | Edit an article (admin) | title, category, body | 200 article |
| DELETE | /advisory/{id} | Delete an article (admin) | none | 200 message |

**Business rules enforced by the API:** an order is rejected with 400 if its quantity is below the listing's minimum order quantity or above the available quantity; accepting an order reduces the listing quantity and marks it `sold_out` at zero; listings with `is_urgent`

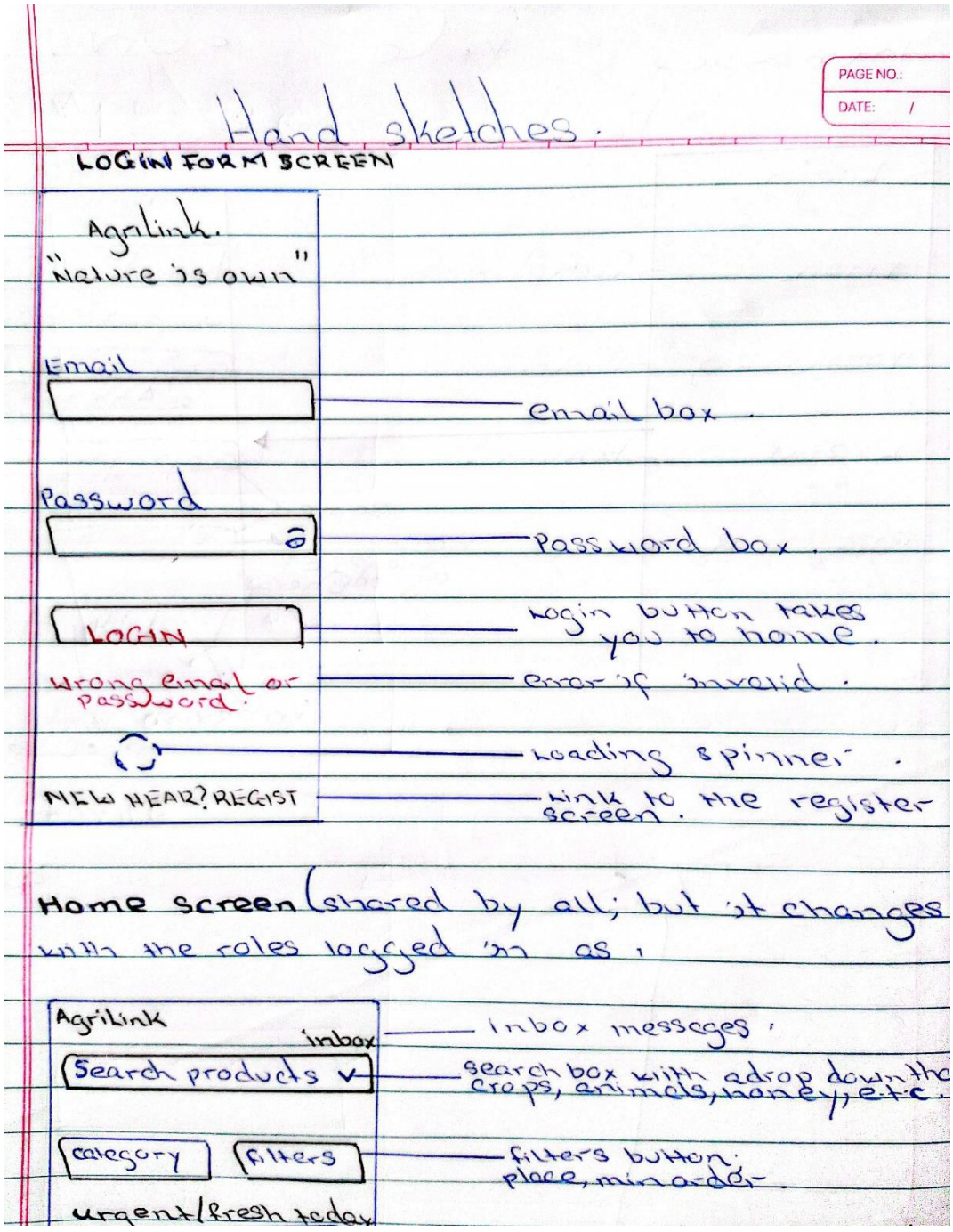
true are returned first; a user can only edit or delete their own listings, orders and messages (otherwise 403); a message can only be read by its sender and receiver.

## 8. Main Screens

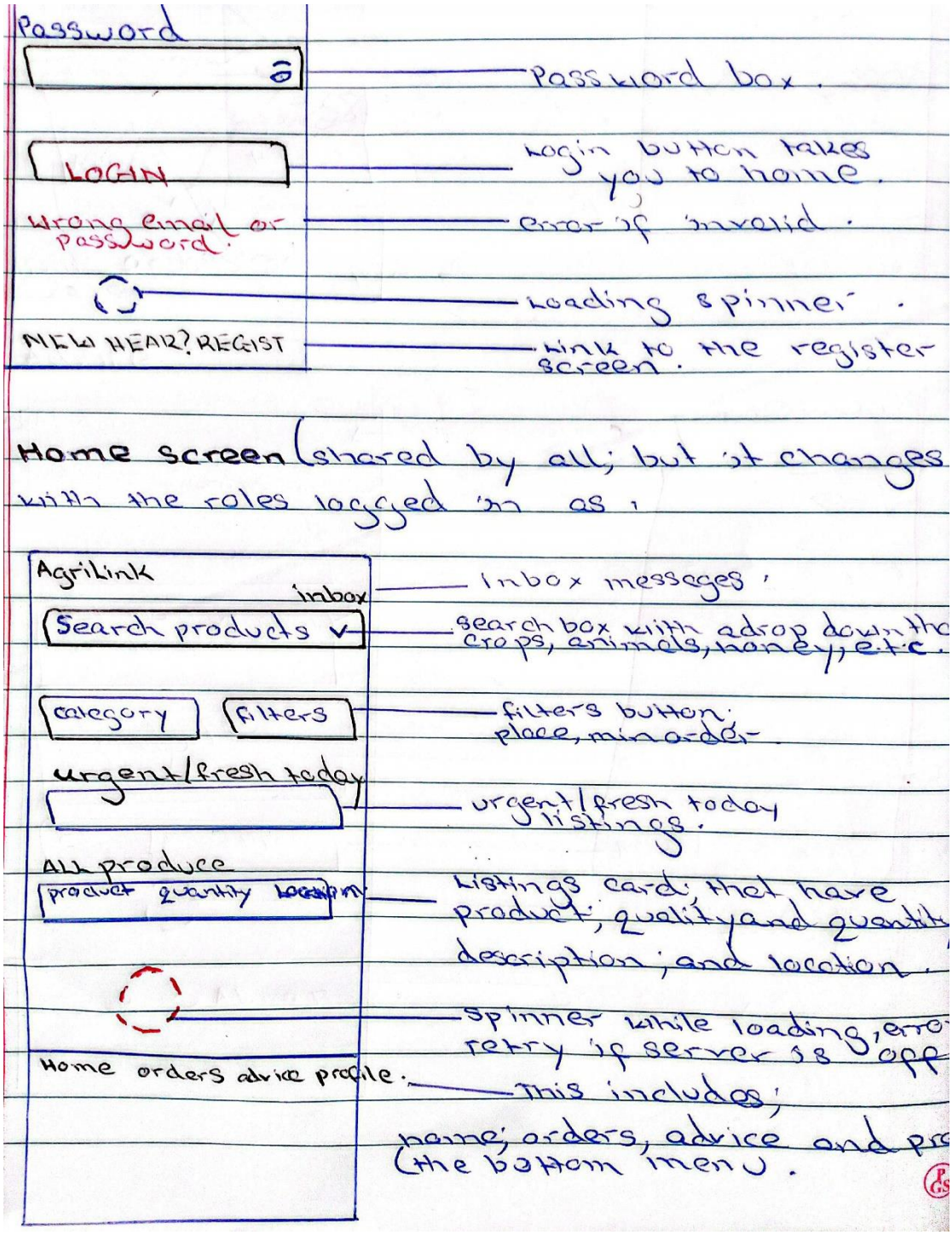
| # | Screen | Used by | Main elements |
|---|---|---|---|
| 1 | Login | All | Email, password, login button, link to register |
| 2 | Register | All | Name, phone, email, password, role, buyer type or farmer type |
| 3 | Home / Listings | All | Search and filter (crop, location, minimum order), Urgent / Fresh Today section, listing cards, loading and error states |
| 4 | Listing Detail | Buyer | Price, minimum order, harvest and best-before dates, quantity box, Place Order and Message Farmer buttons |
| 5 | Order Confirmation | Buyer | Crop, quantity, total price in UGX, confirm and cancel buttons |
| 6 | My Orders | Buyer | Orders with status tag and Cancel button on pending ones |
| 7 | My Listings | Farmer | Own listings with Add, Edit and Delete |
| 8 | Add / Edit Listing | Farmer | Product dropdown, quantity, price, minimum order, dates, urgent switch, validation |
| 9 | Received Orders | Farmer | Orders on own listings with Accept and Reject |
| 10 | Inbox | All | List of conversations |
| 11 | Conversation | All | Messages, text box, Send button |
| 12 | Advisory | All | Category tabs, article list, article detail |
| 13 | Profile | All | Details, edit, Logout button |

| | | | |
|---|---|---|---|
| 14 | Admin Dashboard | Admin | Links to products, advisory and farmer questions |
| 15 | Manage Products | Admin | Product list with Add, Edit, Delete |
| 16 | Add / Edit Product | Admin | Name, category, unit |
| 17 | Manage Advisory | Admin | Article list with Add, Edit, Delete |
| 18 | Add / Edit Article | Admin | Title, category, text, Publish |
| 19 | Farmer Questions | Admin | Messages from farmers with a reply box (extension officer window) |
| | | | |

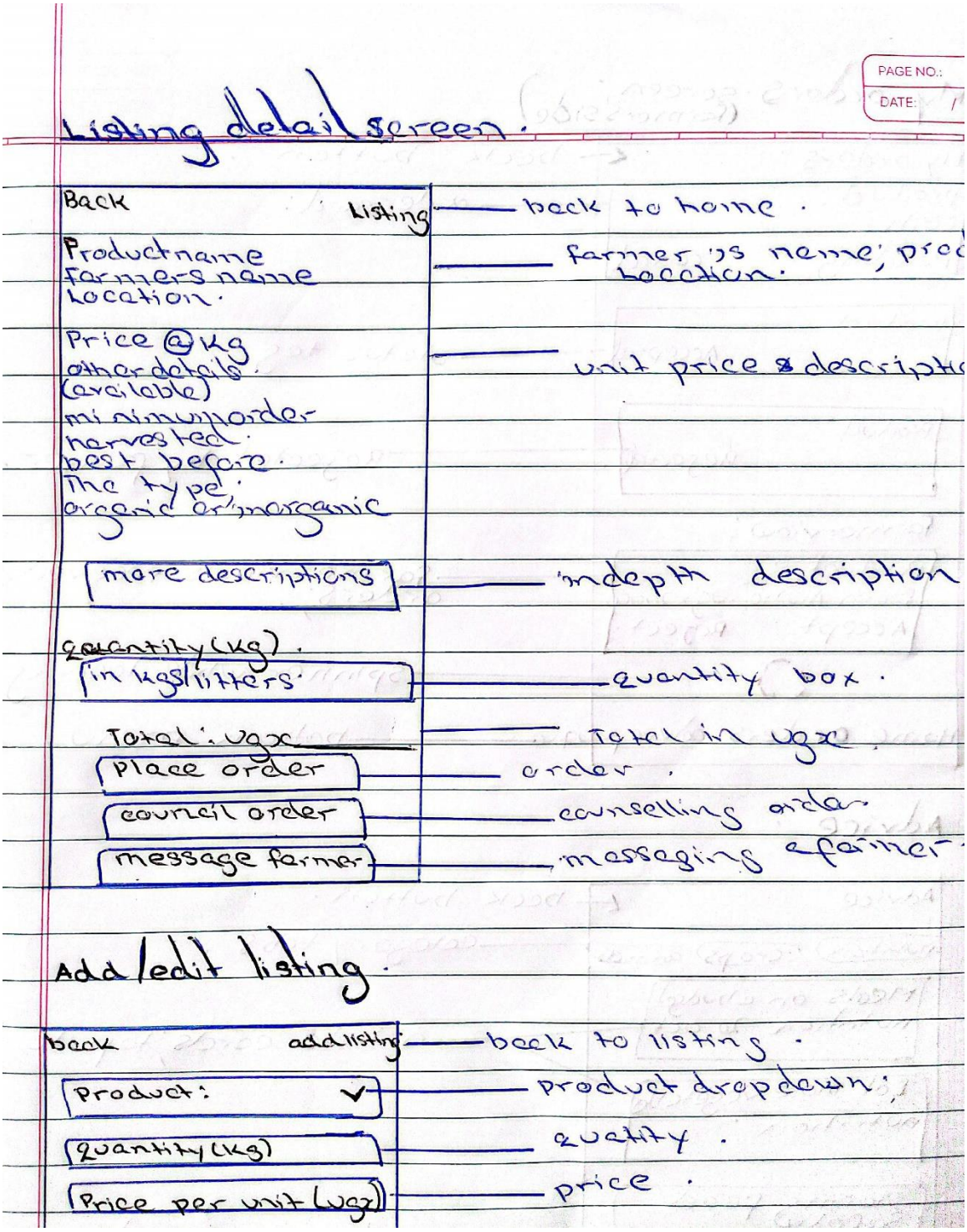
**SCREENS DRAWN BY HAND**



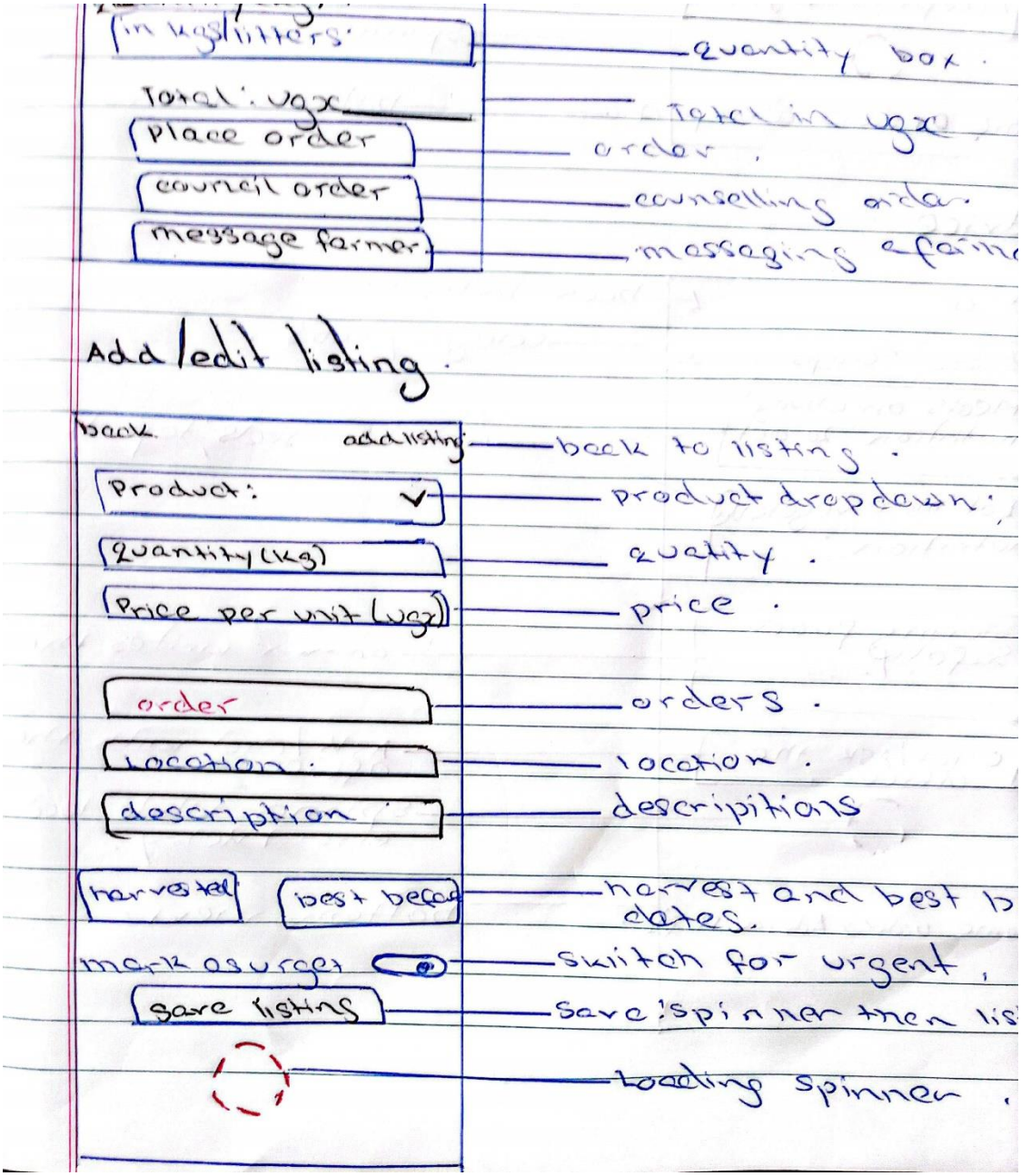

**LOGIN SCREEN DRAWN BY HAND**



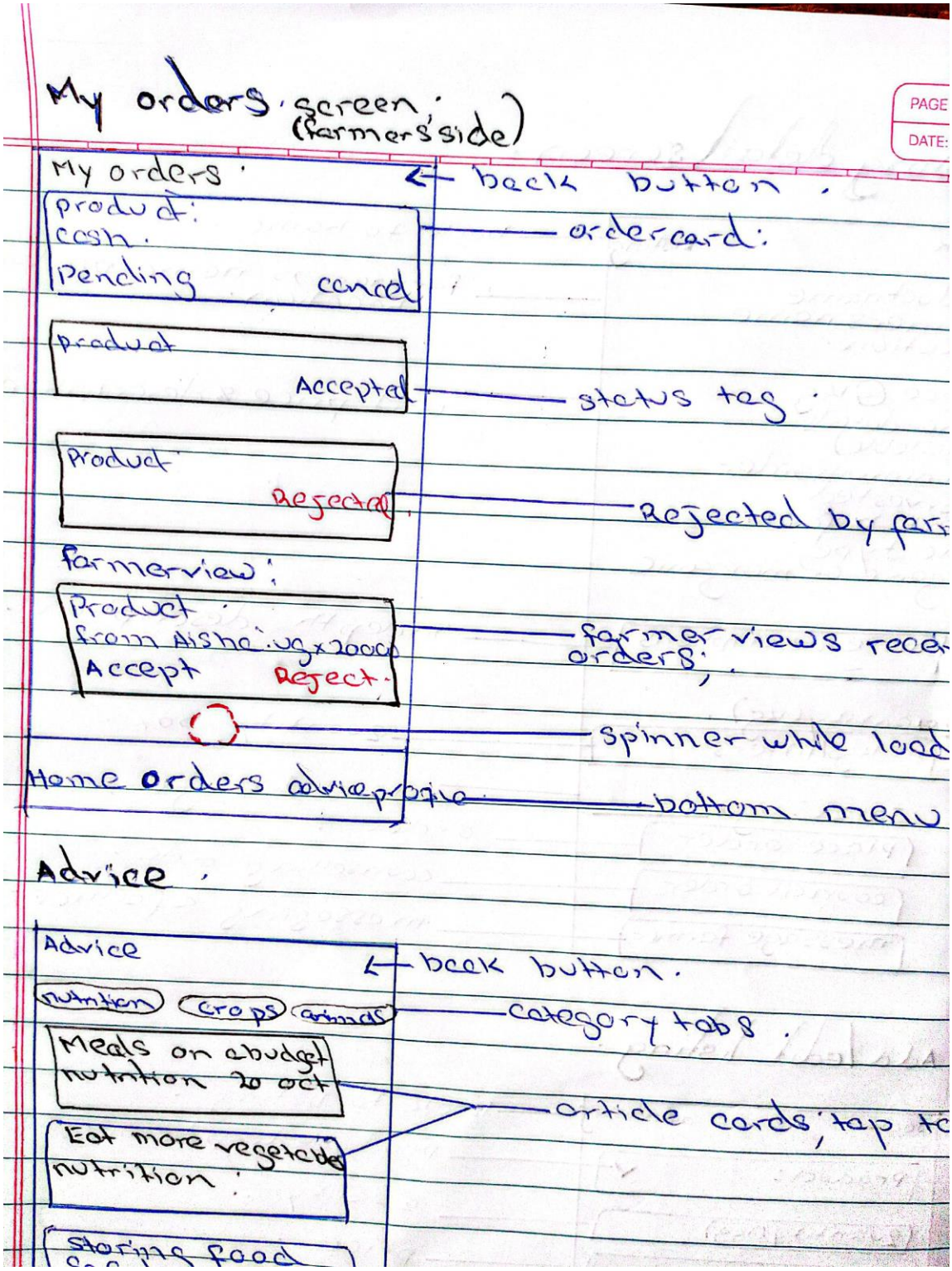

**HOME SCREEN DRAWN BY HAND**



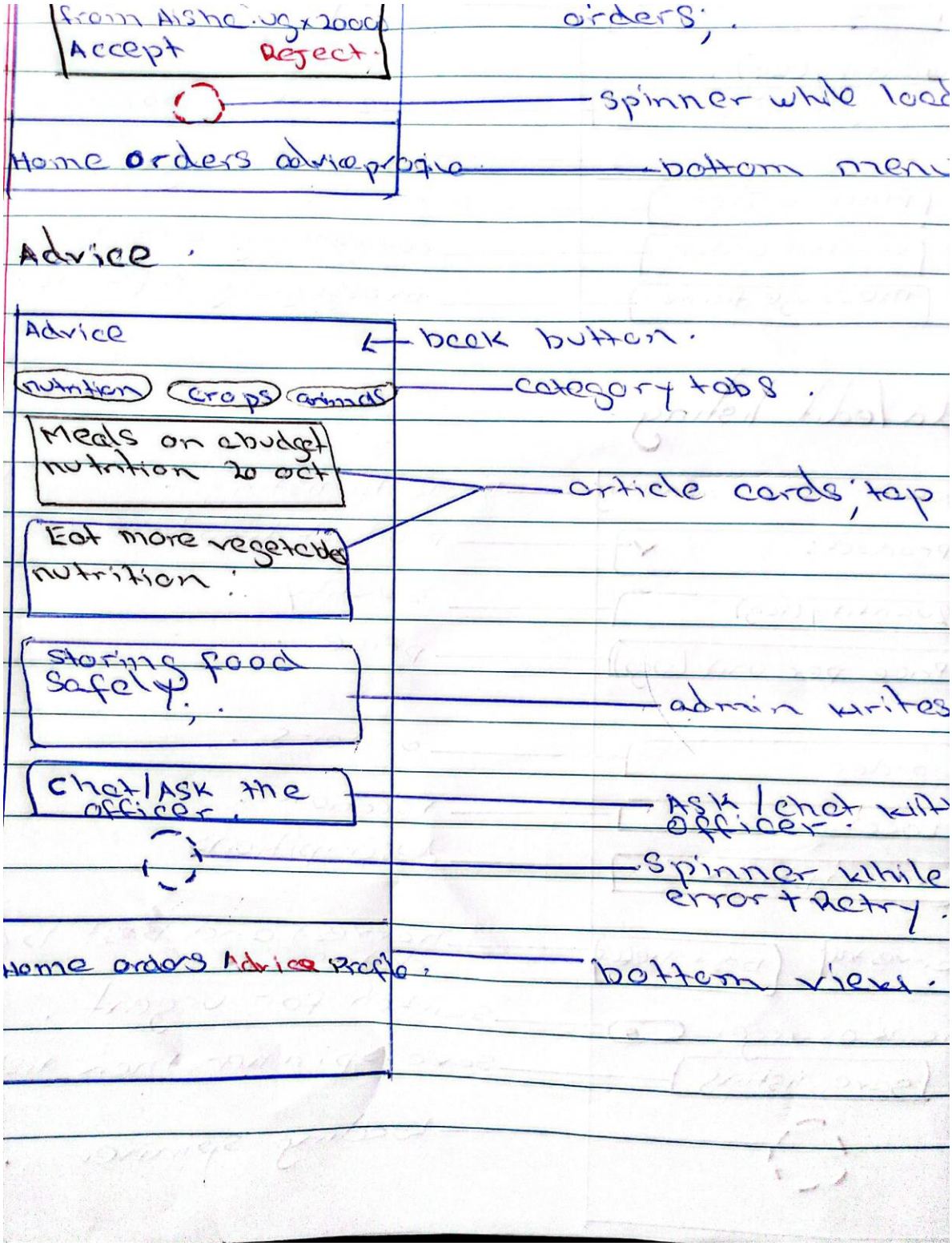

**LISTINGS SCREEN TO THE SIDE OF THE BUYERS**




**FARMERS IS LISTINGS SCREEN**




**ORDERS SCREEN ON THE SIDE OF THE FARMERS**




**ADVISORY SCREEN**

9. Team Roles

| **Member** | **Role** | **Owns** |
|---|---|---|
| MWIWA YUSUF | Backend lead | API, database, run instructions |
| NAKIRANDA HAFISWA | Flutter lead | Screens, navigation, forms |
| NALULE AISHA | Analyst and integration lead | This document, API docs, testing, README |